

Architecture Plan: Vision-Based RL

1. Core Concepts & Terminology

- **Rotary Position Embeddings (RoPE):** Injects relative temporal distances into the attention mechanism. Crucial for the agent to distinguish the order of past observations.
- **Grouped-Query Attention (GQA):** Reduces the KV cache memory footprint. Essential for fitting long sequence rollouts on a single A100 GPU without OOM errors.
- **RMSNorm:** A computationally cheaper alternative to LayerNorm. Stabilizes the massive variance in gradient updates inherent to RL.
- **KV Cache:** Stores previously computed key/value pairs during sequence generation/rollouts to prevent redundant matrix multiplications at every time step.
- **Context-Window Truncation:** Managing memory horizons by passing detached hidden states across chunk boundaries during backpropagation through time (BPTT).

2. Visual Pre-Training Strategies (The CNN)

The goal is to compress raw frames into dense semantic vectors offline. These weights will be frozen before RL begins.

The Convolutional Autoencoder (Reconstruction)

The model compresses the frame into a latent bottleneck, then attempts to reconstruct the exact original pixels using a Mean Squared Error (MSE) loss.

- **Pros:** Simple to implement. Guaranteed to capture all visual information.
- **Cons:** Over-allocates latent capacity to useless high-frequency details (e.g., UI elements, pixel noise) rather than semantic state (e.g., "enemy Pokemon type").

Contrastive Learning (The Modern Approach)

Using frameworks like SimCLR or BYOL. Two augmented crops of the same frame are pulled together in latent space, while different frames are pushed apart via InfoNCE loss.

- **Pros:** Learns semantic invariance. Ignores pixel-perfect reconstruction in favor of understanding "what" is on screen. Highly effective for freezing as an RL feature extractor.
- **Cons:** Requires careful design of data augmentations; overly aggressive crops can destroy necessary UI state information (like HP bars).

Forward Dynamics Modeling (Next-Frame Prediction)

The CNN encodes State t , and a projection head predicts the latent vector for State $t+1$ given Action t .

- **Pros:** Forces the network to learn the "physics" and transition rules of the game engine. Naturally isolates factors of variation that are controllable by the agent.
- **Cons:** Susceptible to the "noisy TV" problem where stochastic environmental changes (like an NPC moving randomly) dominate the loss function.

3. Temporal Sequence Modeling (The Transformer)

Once the CNN weights are frozen, the Transformer processes the latent vectors over a rolling context window.

- **Memory Horizon:** Target a sequence length of 500-1000. This provides enough temporal context to navigate caves and backtrack without exceeding the A100 VRAM limits during PPO rollouts.
- **Architecture integration:** The sequence consists of alternating (or concatenated) combinations of *[Latent State, Action, Reward]*. Apply RMSNorm strictly before attention layers to prevent the RL value-loss spikes from collapsing the attention weights.

4. PPO Reward Function Design

The reward function drives the policy updates. Sparse, long-term goals must be bridged by dense, local incentives.

- **Delta-Based Rewards:** Reward must be strictly positive-delta driven. For example, $\text{reward} = \max(0, \text{current_total_level} - \text{max_historical_total_level})$. This prevents the agent from exploiting cycles (e.g., repeatedly depositing and withdrawing a Pokemon).
- **Intrinsic Curiosity (Exploration Bonus):** Use a hash map of quantized player coordinates. Issue a decaying scalar reward each time a new coordinate is reached.
- **Clipping:** Clamp extreme reward values (e.g., defeating a gym leader) into a fixed range $[-1, 1]$ during the advantage estimation phase to prevent gradient explosion in the actor network.

5. Execution Steps & Architectural Strategy

- **Latent Space Normalization:** Before passing frozen CNN embeddings to the custom transformer blocks, apply an explicit affine normalization layer. PPO value networks are hyper-sensitive to unscaled input variances; raw contrastive latents will cause immediate policy collapse.
- **Sequence Memory Smoke Test:** Validate the architecture in the sub-sim environment first. Construct a deterministic curriculum where a specific visual token is injected at step 0, requiring an action at step 400. This isolates and verifies the RoPE and KV Cache logic before introducing actual visual data.
- **GQA Tuning:** Profile the KV cache memory footprint on the A100 for a sequence length of 1000. Adjust the GQA group size to guarantee the PPO rollout buffer can hold at least 4-8 parallel environment trajectories without triggering memory offload.
- **Truncation Handling:** Implement a rolling state passing mechanism. During PPO updates, instead of zeroing out the Transformer state at the start of a new context chunk, pass the final KV states of chunk $t-1$ as detached tensors to chunk t . This prevents the agent from going blind at arbitrary chunk boundaries.

6. Required Reading (Papers)

- **Contrastive Learning:** "A Simple Framework for Contrastive Learning of Visual Representations" (*SimCLR*) - Chen et al.

- **Contrastive Learning (No Negative Pairs):** *"Bootstrap Your Own Latent - A New Approach to Self-Supervised Learning" (BYOL)* - Grill et al.
- **Forward Dynamics / RL:** *"Curiosity-driven Exploration by Self-supervised Prediction" (ICM)* - Pathak et al.
- **Sequence Architectures:** *"Decision Transformer: Reinforcement Learning via Sequence Modeling"* - Chen et al.
- **Transformer Optimizations:** *"RoFormer: Enhanced Transformer with Rotary Position Embedding"* (RoPE) and *"GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints"*.
- **RL Fundamentals:** *"Proximal Policy Optimization Algorithms"* - Schulman et al.